



Universidad Autónoma de Querétaro
Facultad De Ingeniería



**FACULTAD DE
INGENIERÍA**

Problema de horarios de cursos universitarios (UCTP)

Algoritmos Metaheurísticos Tarea 4

P R E S E N T A

Ing. Daniel Eduardo González Alvarado

Profesor

Dr. Marco Antonio Aceves Fernández

17 de Noviembre de 2025

Índice

1. Introducción	1
1.1. UCTP	1
1.2. Cursos Considerados	1
1.3. Consideraciones	2
2. Desarrollo	3
2.1. Representación del Cromosoma	3
2.2. Datos y Restricciones Incorporadas	3
2.3. Generación de Individuos	4
2.4. Función de Aptitud	4
2.5. Operadores Genéticos	4
2.6. Construcción del Horario Final	5
2.7. Ejecucion del AG	5
3. Resultados	5
4. Discusión	6
5. Conclusión	6

1. Introducción

1.1. UCTP

La asignación automática de horarios académicos, conocida como University Course Timetabling Problem (UCTP), es un problema clásico de optimización combinatoria presente en instituciones educativas de todos los niveles. Consiste en asignar, de manera eficiente, cursos, grupos, salones y franjas horarias, respetando un conjunto de restricciones duras (hard constraints), tales como evitar traslapes de clases, asignación de salones adecuados o disponibilidad de espacios, así como restricciones blandas (soft constraints), que buscan mejorar la calidad del horario, como minimizar demasiadas horas consecutivas, distribuir las clases durante la semana o evitar largos periodos sin actividad.

El UCTP es un problema NP-hard debido al gran número de combinaciones posibles y a la interacción compleja entre todas las restricciones, lo que hace inviable una solución óptima mediante métodos exhaustivos. Por ello, se han adoptado algoritmos metaheurísticos capaces de explorar grandes espacios de búsqueda de manera eficiente. Entre ellos, los Algoritmos Genéticos (AG) han demostrado ser una herramienta robusta para encontrar soluciones de alta calidad en tiempos razonables.

Los AG se inspiran en los mecanismos de la evolución biológica, operando con una población de posibles soluciones (individuos) que evolucionan a través de operadores como selección, cruce y mutación. En el contexto del UCTP, cada individuo representa un horario completo con todas las materias asignadas y su calidad se evalúa mediante una función de aptitud que penaliza el incumplimiento de restricciones. A través de múltiples generaciones, el algoritmo incrementa la viabilidad y calidad de los horarios generados, permitiendo obtener soluciones que cumplen con las restricciones duras y optimizan las blandas.

Abordaremos una implementación del UCTP utilizando un Algoritmo Genético diseñado para manejar múltiples planes de estudio, diversos tipos de salones y diferentes duraciones de clase. Además, se incorporan mecanismos adicionales como verificación de compatibilidad de salones, penalización de cruces, control de consecutividad máxima de clases y exportación automatizada de los horarios en formato Excel para su inspección. Este enfoque permite generar horarios completos y coherentes ajustados a las necesidades reales de operación académica.

1.2. Cursos Considerados

Para este problema nos enfocaremos en 4 cursos diferentes los cuales son:

- Maestría en Ciencias en Inteligencia Artificial.
- Ingeniería Física.
- Ingeniería en Nanotecnología.
- Ingeniería en Biomedica.

En este caso cada curso cuenta con su propio plan de estudio, pero todas comparten los mismos salones disponibles así como los laboratorios, además de que en algunos casos aunque las materias sean las mismas, se deben impartir por curso no solo una vez, por lo cual también se toma en cuenta que el tamaño de los grupos corresponda con el cupo del salón.

1.3. Consideraciones

Para este ejercicio se tomaron como referencia los salones disponibles en el Campus Aeropuerto de la UAQ y que una vez que se recolecto la informacion de los salones se obtuvo lo siguiente:

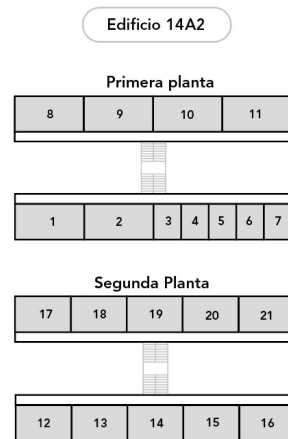


Figura 1: Salones Disponibles en el Edificio 14A2

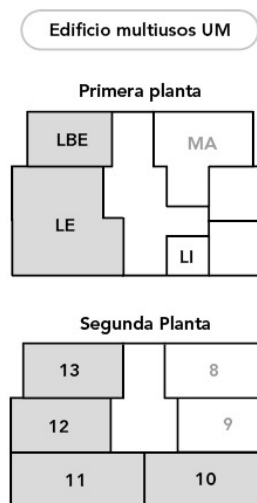


Figura 2: Salones Disponibles en el Edificio de Usos Múltiples

Como podemos ver en el Edificio 14A2 (Figura 1) existen salones más pequeños los cuales se deben tomar en cuenta debido a su cupo. Por otro lado en el Edificio de Usos Múltiples (Figura 2) es donde existen dos de los laboratorios disponibles, sin embargo despues de analizar el plan de estudios de los diferentes cursos se tomo la libertad de crear un laboratorio extra de Química para las carreras que lo necesiten, este laboratorio si existe, simplemente se encuentra en otro edificio.

2. Desarrollo

Partimos de la obtención de nuestros datos que nos servían para codificar y generar nuestros individuos por lo cual se hizo una base de datos investigando el plan de estudios de cada curso considerado para este trabajo los cuales se mencionaron en la sección anterior. 1.2

La resolución del UCTP mediante Algoritmos Genéticos requiere definir una representación clara del problema, así como los operadores evolutivos y una función de aptitud capaz de satisfacer las restricciones duras y optimizar las restricciones blandas.

2.1. Representación del Cromosoma

Cada individuo de la población representa un horario completo. Para ello, se construyó un cromosoma donde cada gen corresponde a un evento académico (materia-grupo) definido en la base de datos. Cada gen se codifica como un par ordenado:

$$\text{gen}_i = (\text{room_index}_i, \text{start_slot}_i)$$

donde:

- **room_index** identifica el salón asignado, indexado en la tabla de salones disponibles.
- **start_slot** representa el bloque de tiempo de inicio en la grilla semanal.

Esta representación permite manejar clases de diferente duración, tipos de aula (laboratorio y salón de clases), y múltiples cursos simultáneamente. Además, facilita la verificación de cruces, compatibilidad y duración sin necesidad de estructuras intermedias complejas.

2.2. Datos y Restricciones Incorporadas

La base de datos utilizada contiene:

- Lista de cursos, su duración en bloques y el número de sesiones por semana.
- Relación curso-grupo para evitar combinar grupos distintos en un mismo curso.
- Tipos de salón requeridos (normal, laboratorio).
- Tabla de salones disponibles con su *room_type* correspondiente.
- Número total de bloques de tiempo por día y por semana.

A partir de estas tablas se construyeron estructuras auxiliares como:

- `compatible_rooms[i]`: lista de salones posibles para el evento *i*.
- `duration_slots[i]`: número de bloques que ocupa cada clase.

2.3. Generación de Individuos

Para crear la población inicial, cada evento se asigna de la siguiente manera:

1. Selección aleatoria de un salón dentro de `compatible_rooms`.
2. Selección aleatoria del `start_slot`, asegurando que la clase completa quepa en el día.
3. Verificación básica para evitar cruces evidentes dentro del mismo individuo.

Este proceso genera soluciones diversas que cumplen mínimamente con las restricciones duras estructurales.

2.4. Función de Aptitud

La función de aptitud se define como la suma de penalizaciones por incumplimiento de restricciones. Esto permite mantener una formulación flexible y ajustable. Las penalizaciones actuales incluyen:

- **Cruce de dos clases en el mismo salón y mismo bloque** (penalización alta).
- **Uso de un salón incompatible** con el tipo requerido.
- **Clases que exceden el día** por mala asignación de *start_slot*.
- **Más de dos horas consecutivas**: si un grupo tiene tres o más clases seguidas de la misma materia, se aplica penalización creciente.
- **Distribución semanal deficiente**: cuando un curso concentra todas sus horas en un mismo día.

La aptitud total se define como:

$$\text{fitness}(ind) = \sum_j \text{penalización}_j(ind)$$

y el objetivo del algoritmo es minimizarla.

2.5. Operadores Genéticos

Se emplean los operadores clásicos adaptados a la estructura del horario:

- **Selección por torneo**: favorece individuos con menor penalización.
- **Cruce uniforme por gen**: cada gen del hijo proviene aleatoriamente de uno de los padres, manteniendo la pareja (salón, inicio).
- **Mutación controlada**: se permite cambiar el salón o el bloque de inicio respetando las compatibilidades.

Estos operadores permiten explorar el espacio de soluciones sin destruir completamente buenas configuraciones parciales.

2.6. Construcción del Horario Final

Una vez que el AG converge a una solución con baja penalización, se utiliza una función de decodificación para transformar el cromosoma en un horario estructurado por día. Para cada día, se genera automáticamente:

- Una matriz con bloques de tiempo en el eje vertical.
- Los salones disponibles en el eje horizontal.
- Las clases asignadas en cada celda con su nombre y grupo.

Finalmente, se exporta a un archivo Excel donde cada hoja representa un día de la semana. Este archivo también incorpora formatos de color para distinguir carreras facilitando el análisis visual de los resultados, los colores asignados fueron:

- MC. Inteligencia Artificial en azul
- Ing. Nanotecnología en amarillo
- Ing. Biomedica en verde
- Ing. Física en morado
- Cruces en rojo.

2.7. Ejecucion del AG

Con todo lo anterior menc

3. Resultados

Podemos observar que con las restricciones de conjuntos (i, j) las rutas se hacen mas complejas de determinar, esto se debe a que ya no es tan facil tener combinaciones validas como aptas, y a su vez esto puede verse en resultados como en la figura ?? donde existe un “cruce” en la ruta sin embargo esto aunque pareciera extraño el resultado general es bastante bueno al encontrarse entre los mejores obtenidos. Podemos observar nuestros resultados ordenados de la siguiente manera del mejor al peor:

1. 4,415,91Km PBX 1000 – ??
2. 4,607,21Km PBX 500 – ??
3. 4,610,89Km PMX 1000 – ??
4. 4,677,34Km PMX 100 – ??
5. 4,905,76Km PMX 500 – ??
6. 4,927,89Km PBX 100 – ??

Como podemos ver la mejor ruta se encontro con el metodo PBX y poblacion de 1000 esta combinacion se ve favorecida debido al espacio de busqueda inicial ya que tenemos una poblacion muy amplia. Por otro lado la segunda mejor ruta la encontramos con el mismo metodo y una poblacion más pequeña esto demuestra el buen desempeño de este metodo ya que en ambos casos tambien observamos la rapida convergencia del algoritmo en ambos casos siendo alrededor de las 150 generaciones.

4. Discusión

Con los resultados obtenidos podemos determinar que el TSP es una gran ejercicio para probar la calidad de nuestros AG debido a su versatilidad asi como practicidad en casos reales. En este ejercicio podemos ver que estas restricciones nos llevan a soluciones que estan mas fuera de nuestra vision, es decir combinaciones distintas a lo que esperaríamos, ya que en el caso del TSP sencillo, las mejores rutas se encontraban al hacer un circuito, pero en esta ocasion podemos ver que las rutas generan distintas formas por lo que tambien notamos que existen un conjunto variado de combinaciones, no solo un conjunto con las mismas características.

Para nuestra mejor ruta en la seccion ?? podemos ver que aunque el mapa de la figura ?? muestra el cierre de la ruta esto solo es una convencion ya que la ruta en si es similar a una “S” llendo de norte a oeste, luego este y por ultimo al sur, esta ruta demuestra estos casos donde la mejor opcion no es un circulo como tal, si no las distancias más cortas.

5. Conclusión

Con base en lo mencionado a lo largo de este reporte asi como en el del TSP simple podemos ver que este ejercicio ofrece un muy buen campo de pruebas para nuestros AG. Para su aplicacion en el mundo real es fundamental tener en cuenta los casos como el que abordamos en este ejercicio donde existen limitantes a las rutas que podemos crear, ya que no siempre nos encontraremos con el camino “ideal” por diversos factores, como clima, caminos en reparacion, rutas cerradas por personas o problemas con el terreno y que aunque no se abordó la situacion tambien podemos encontrar escenarios donde sea obligatorio cumplir con sub-rutas obligatorias, es decir que exista una cadena dentro de nuestro recorrido que tenga que ser visitada en ese orden. Esto vas vez va limitando nuestras opciones asi como aumenta la complejidad del ejercicio, sin embargo nos muestra un panorama más real de lo que podemos ver en el mundo real.

Por otro lado aunque en este caso nos enfocamos en visitar ciudades podemos llevar este mismo ejercicio a otros ambitos, por ejemplo rutas de transporte publico, donde se tome en cuenta todas las estaciones donde se detienen los servicios de transporte, desde autobuses hasta trenes. E incluso podriamos efectuar un AG que nos muestre varias rutas diferentes para cubrir un espacio más amplio. Otro ejemplo de estas aplicaciones podria ser en el diseño de circuitos electronicos donde se busque el mejor diseño para eficientar la manofactura en masa de estas tarjetas, pasando por puntos especificos para los componentes asi como restricciones de que no choquen entre si.

Con esto en mente la aplicacion de los AG como herramienta de optimizacion no se limita solo a los ejercicios comunes que se abordan en clase si no en el mundo real, un caso real de esto lo podemos ver en la cadena McDonald’s donde en 1948 los hermanos inventaron el “Speedy

Service System” que optimizó el proceso de producción mediante la especialización de tareas, y redujo drásticamente el tiempo de preparación de 30 minutos a solo 30 segundos, entre otras cosas esto se hizo al reorganizar el espacio de la cocina para simular una línea de montaje. Esto los colocó rápidamente como un exponente en el mundo de la comida rápida. Partiendo de este caso la misma lógica podríamos aplicarla para tener espacios de trabajo más óptimos en los negocios de servicio.